

부르릉쾅쾅

남윤진, 박동수

한국공학대학교 게임공학과

{nyj121, qkrdlf8219}@tukorea.ac.kr

요 약

본 프로젝트는 아이템과 대시 시스템을 활용하여 전략적인 플레이가 가능한 멀티플레이 레이싱 게임을 개발하는 것을 목표로 하였다. 플레이어의 주행 실력뿐만 아니라 순간적인 판단과 아이템 활용이 승패에 영향을 미치도록 설계하였으며, 최대 4명의 플레이어가 동시에 경쟁할 수 있는 환경을 구현하였다. DirectX12 기반의 렌더링 파이프라인을 구성하고 네트워크를 활용한 실시간 멀티플레이 시스템을 개발하여 플레이어 간 동기화를 수행하였다. 또한 게임 내 시각 효과를 효율적으로 관리하고 재사용할 수 있도록 이펙트 라이브러리를 설계·구현하였다. 이를 통해 렌더링, 네트워크, UI 및 이펙트 시스템 등 게임 개발의 핵심 기술을 범용 게임 엔진 없이 구현하고 통합하였다.

1. 서 론

캐주얼한 멀티플레이 레이싱 환경을 제공하고자 하였다.

1.1 게임 제작 목적: 본 프로젝트는 단순한 속도 경쟁을 넘어, 아이템과 대시 시스템을 활용한 전략적 판단이 승패에 영향을 미치는 멀티플레이 레이싱 게임 개발을 목표로 하였다. 또한 최대 4명의 플레이어가 실시간으로 경쟁할 수 있는 환경을 구축하여 경쟁의 재미를 제공하고자 하였다.

1.2 게임 기획 의도: 기존 레이싱 게임은 주행 실력에 따라 승패가 결정되는 경우가 많아 실력 차이가 큰 플레이어 간 경쟁에서 재미가 감소할 수 있다. 이를 보완하기 위해 다양한 아이템과 대시 시스템을 도입하여 순간적인 판단과 전략적 선택이 경기 결과에 영향을 줄 수 있도록 설계하였다. 또한 누구나 쉽게 익힐 수 있는 조작 방식을 적용하여 초보자 and 숙련자가 함께 경쟁하며 즐길 수 있는

1.3 게임 주요 특징: 본 게임은 DirectX 12 기반의 3D 멀티플레이 레이싱 게임으로, 대시 및 아이템 시스템을 활용한 전략적인 플레이를 지원한다. 또한 다양한 시각 효과를 효율적으로 관리하고 재사용할 수 있도록 자체 제작 이펙트 라이브러리를 설계 및 구현하였으며, 이를 실제 게임에 적용하였다. 렌더링 시스템은 DirectX 12 기반으로 구축하였고, 네트워크 통신을 통해 최대 4명의 플레이어가 실시간으로 경쟁할 수 있는 멀티플레이 환경을 구현하였다.

2. 개발 내용

2.1 게임 설계: 본 게임은 최대 4인이 참여

하는 멀티플레이 레이싱 게임으로, 플레이어는 차량을 직접 조작하여 지정된 트랙을 3바퀴 완주하며 순위를 겨룬다. 주행 중에는 대시 기능과 아이템을 활용해 자신의 주행 성능을 높이거나 상대를 방해할 수 있으며, 모든 플레이어가 완주하면 최종 순위가 집계된다.

게임은 주행 및 대시 시스템, 이펙트 시스템, 아이템 시스템, 멀티플레이어 시스템을 중심으로 설계하였으며, 각 시스템이 유기적으로 연동되도록 구성하였다.

2.2 개발 환경 및 방법론: 본 게임은 Microsoft Visual Studio 환경에서 C++로 개발하였다. 그래픽 렌더링은 DirectX 12를 기반으로 구현하였으며, 사용자 인터페이스는 D3D11On12와 Direct2D를 활용하여 제작하였다.

버전 관리는 GitHub를 통해 수행하였으며, 팀원 간 코드 공유와 협업에 활용하였다. 개발 과정에서는 애자일(Agile) 방식을 적용하여 기능 단위로 구현과 테스트를 반복적으로 수행하였으며, 플레이 테스트 결과를 반영하여 시스템을 지속적으로 개선하였다.

2.3 주행 및 대시 시스템: 주행 시스템은 차량의 이동, 가속 및 감속 기능을 기반으로 구현하였다. 차량의 현재 속도와 최대 속도를 별도로 관리하여 자연스러운 속도 변화를 구현하였고, 충돌 발생 시 너백과 제어 제한을 적용하여 충돌에 대한 패널티를 부여하였다.

대시 시스템은 게이지 기반 자원 관리 방식으로 구현하였다. 플레이어가 대시를 사용하면 게이지가 지속적으로 소모되며 차량의 최대 속도가 증가한다. 반대로 대시를 사용하지 않을 경우 게이지가 자동으로 회복되도록 구

현하였다.

또한 게이지가 모두 소모되면 과열 상태가 발생하도록 설계하였다. 과열 상태에서는 일정 시간 동안 대시 사용이 제한되며 차량의 속도가 감소하여 무분별한 대시 사용을 방지하였다.

경기 밸런스 유지를 위해 현재 순위에 따라 대시 게이지 회복 속도를 차등 적용하는 보정 시스템을 구현하였다. 후순위 플레이어일수록 더 높은 게이지 회복량을 제공하여 선두 플레이어와의 격차를 완화하고, 경기 후반까지 경쟁이 유지될 수 있도록 설계하였다.

2.4 아이템 시스템: 아이템 시스템은 경기 중 획득한 아이템을 사용하여 다양한 효과를 얻을 수 있도록 구현하였다. 플레이어는 트랙에 배치된 아이템 박스를 통해 아이템을 획득하며, 원하는 시점에 사용할 수 있다.

구현된 아이템은 대시 게이지 충전, 최고 속도 증가, 대시 게이지 무제한 사용, 상대 플레이어의 대시 제한의 네 가지로 구성된다. 각 아이템은 경기 상황에 따라 전략적으로 활용할 수 있으며, 단순한 주행 실력뿐 아니라 순간적인 판단과 선택이 경기 결과에 영향을 미치도록 설계하였다.

[표 1] 아이템 종류와 속성

아이템	속성
대시 게이지 충전	대시 게이지 30 증가
최고 속도 증가	3초간 최고 속도 25 증가
무한 대시	3초간 대시 게이지 소모 없음
대시 봉인	3초간 대시 시스템 봉인

2.5 이펙트 라이브러리 구현: 본 프로젝트에서는 다양한 시각 효과를 효율적으로 관리하고 재사용하기 위해 DirectX 12 기반의 자체 이펙트 라이브러리를 설계 및 구현하였다.

기존에는 게임마다 이펙트 관련 코드를 개별적으로 작성해야 했으나, 이러한 방식은 코드 중복이 증가하고 유지보수가 어려운 문제가 있었다. 이를 해결하기 위해 이펙트 기능을 독립적인 라이브러리 형태로 분리하여 다른 프로젝트에서도 재사용할 수 있도록 설계하였다.

이펙트 라이브러리는 Singleton 구조를 기반으로 구현하였으며, Effect Type 열거형을 통해 다양한 이펙트를 통합 관리할 수 있도록 구성하였다. 또한 이펙트 생성 요청을 이벤트 큐에 저장한 후 순차적으로 처리하는 구조로 이펙트 생성과 렌더링 과정을 분리하였다.

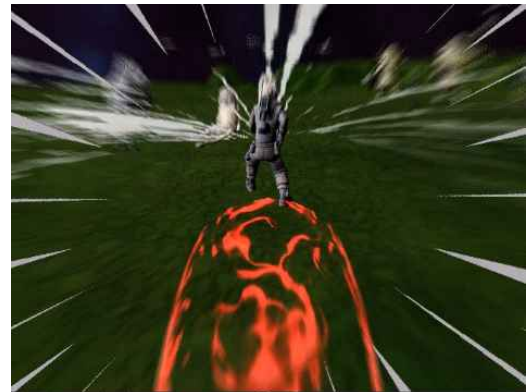
라이브러리는 파티클 기반 이펙트와 메시 기반 이펙트를 모두 지원하도록 설계하였다. 각 이펙트는 위치, 크기, 색상, 지속 시간 등의 정보를 공통 데이터 구조로 관리하며, 동일한 인터페이스를 통해 생성 및 제어할 수 있도록 구현하였다. 또한 Object Pooling 기법을 적용하여 이펙트 객체를 반복적으로 생성 및 삭제하지 않고 재사용함으로써 메모리 할당 비용을 줄이고 성능을 향상시켰다.

구현된 라이브러리는 대시 효과, 충돌 효과, 속도 효과, 아이템 사용 효과 등 다양한 게임 내 연출에 적용하였으며, 실제 게임 프로젝트 뿐만 아니라 DirectX 12 기반 프레임워크를 사용하는 다른 프로젝트에서도 정상적으로 동작하는 것을 확인하였다. 이를 통해 라이브러리의 독립성과 재사용 가능성을 검증하였다.

라이브러리 사용 시 파티클 이펙트는 Play() 함수를 통해 생성할 수 있으며, 이펙

트 종류(EFFECT_TYPE), 위치(Position), 크기(Size), 색상(Color) 등의 파라미터를 전달하여 다양한 연출을 구현할 수 있다. 또한 지속적으로 유지되는 부스터 이펙트의 경우 ToggleLocalBooster() 함수를 통해 활성화 여부와 크기, 색상, 위치 오프셋 등을 제어할 수 있도록 설계하였다. 이를 통해 동일한 인터페이스만으로 다양한 파티클 및 메시 기반 이펙트를 일관된 방식으로 렌더링할 수 있다.

본 라이브러리의 구조 및 적용 방법에 대한 상세 내용은 별도로 첨부한 「이펙트 라이브러리 매뉴얼」에 기술하였다.



[그림 1] 속도선과 부스터 이펙트



[그림 2] 충돌 이펙트



[그림 3] 흠먼지 이펙트



[그림 4] 아이템 사용 이펙트



[그림 5] 아이템 획득 이펙트

2.6 멀티플레이어 구현: 멀티플레이어 시스템은 C++의 Winsock2를 활용한 릴레이 서버 구조를 기반으로 구현하였다. 특정 클라이언트가 서버 역할을 겸하는 대신, 전용 서버를 가동한 뒤 각 플레이어가 해당 서버의 IP 주소를 입력해 로비에 접속하는 방식을 채택하여 게임 로직 처리와 네트워크 부하를 분리했다. 이를 통해 다수의 플레이어가 참가하는 환경에서도 안정적인 실시간 동기화를 제공하도록 설계했다.

네트워크 통신은 TCP 소켓과 select 모델을 활용하여 구현했다. 게임 진행 중에는 플레이어의 좌표, 회전, 주행 속도와 같은 기본 상태 정보뿐만 아니라 대시 게이지 상황, 아이템 획득 및 사용, 이펙트 발생 위치 등을 실시간으로 브로드캐스트하여 모든 참가자가 지연 없이 동일한 게임 상태를 공유할 수 있도록 했다.

또한 로딩 완료 여부, 레이스 시작 카운트다운, 아이템 선점 및 유저 간 충돌 물리 연산과 같은 핵심 이벤트는 서버가 중앙에서 동기화 패킷을 처리하여 일관성을 유지한다. 경기가 종료되면 서버 측에서 각 플레이어의 완주 시간을 취합해 최종 순위를 계산하고, 이를 모든 클라이언트에게 전송하여 일괄적으로 결과 화면을 출력하도록 구현하였다.

3. 결 론

3.1 개발 작품 요약: 본 프로젝트에서 개발한 '부르릉광광'은 DirectX 12 그래픽 API와 Winsock2 소켓 API를 기반으로 자체 프레임워크 환경에서 구축된 3D 자동차

모델 중심 실시간 멀티플레이어 레이싱 게임이다.

상용 엔진에 의존하지 않고 로우레벨 API를 직접 제어하여 그래픽 파이프라인과 네트워크 아키텍처를 설계하였다. 그래픽스 측면에서는 효과적인 이펙트 라이브러리를 구축하여 대시 부스터 효과, 먼지 파티클, 자물쇠 오빗(Orbit) 효과 등을 포스트 프로세싱 기법과 결합해 레이싱 특유의 역동성을 극대화했다. 네트워크 측면에서는 독립형 릴레이 서버를 도입하고 select 모델을 채택하여, 최대 4인의 플레이어가 동시 접속한 환경에서도 대시 게이시 상황, 실시간 위치 및 회전 정보, 아이템 상자 리스폰 및 선점 처리, 충돌 물리 반작용 연산 등을 실시간으로 동기화하는 데 성공했다.

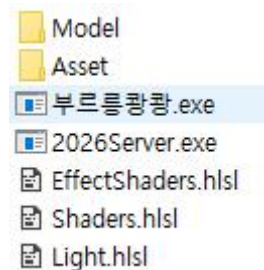
3.2 개발 작품의 장단점: 본 작품은 상용 엔진에 의존하지 않고 DirectX 12와 C++를 활용해 렌더링 파이프라인과 이펙트 시스템을 구조적으로 분리한 커스텀 프레임워크를 성공적으로 구축했다는 점에서 큰 의의가 있다. 특히 먼지, 부스터, 충돌 이펙트, 속도선 등 다양한 파티클과 메시 이펙트를 전달하여 관리하는 이펙트 라이브러리를 모듈화하여 개발함으로써, 향후 다른 프로젝트에서도 코드를 쉽게 재활용할 수 있는 확장성 높은 아키텍처를 완성한 것이 본 프로젝트의 가장 큰 장점이다. 하지만 프레임워크 구축과 네트워크 동기화 등 코어 프로그래밍에 개발 역량을 집중하다 보니 콘텐츠와 시각적인 디테일 측면에서 일부 아쉬움이 남는다. 현재 구현된 아이템의 종류가 제한적이어서 게임 플레이의 다양성이 다소 부족하다는 점이 있다. 또한 맵 트랙과 이펙트에

사용된 에셋 파일의 퀄리티가 다소 투박하여 전반적인 시각적 완성도가 떨어진다는 단점이 있다.

3.3 향후 추가 개발 계획: 3D 레이싱의 역동성에 걸맞은 고품질의 '코믹 및 캐주얼'풍 에셋으로 맵 배경과 파티클 이펙트, UI 리소스를 전면 리뉴얼하여 게임 전체의 아트워크 톤앤매너를 매력적으로 통일할 계획이다. 아울러 부족한 플레이 볼륨을 채우기 위해 아이템의 종류를 더욱 다양화하고, 맵 기믹을 추가 구현함으로써, 캐주얼 레이싱 특유의 깊이와 시각적 즐거움을 모두 갖춘 완성도 높은 게임으로 업그레이드해 나갈 예정이다.

4. 부 록

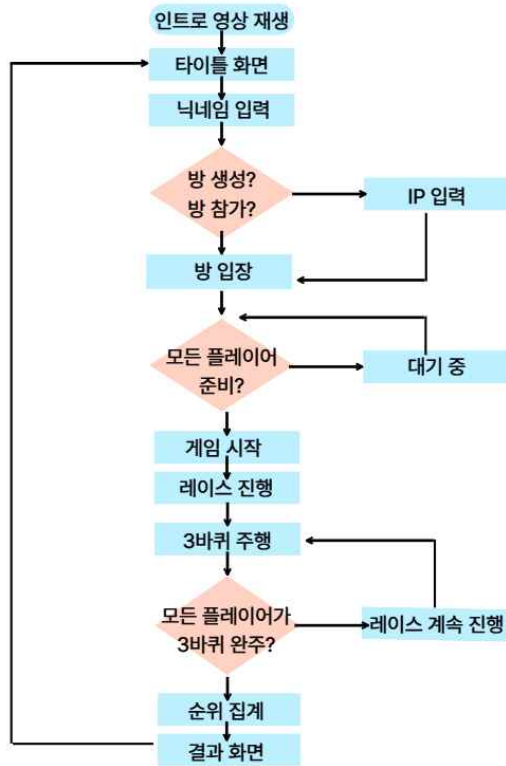
4.1 게임 설치 방법



[그림 6] 실행 파일 구성

1. 제출된 남윤진_박동수_실행파일.zip 파일의 압축을 해제한다.
2. 생성된 폴더 내에서 방을 생성할 플레이어는 2026Server.exe를 먼저 실행한다.
3. 이후 모든 플레이어는 부르릉광광.exe를 실행한다.

4.2 플레이 흐름도:



[그림 7] 플로우 차트



[그림 9] IP주소 입력 화면

방 참가 시 서버 IP 주소를 입력하여 게임에 접속한다.



[그림 10] 메인 대기실 화면

대기방에 접속한 모든 플레이어가 준비 상태를 완료하면, 방장은 게임 시작 기능을 통해 경기를 시작할 수 있다.

4.3 주요 화면 구성:



[그림 8] 메인 로비 화면

유저는 닉네임 입력 후 (미입력시 기본 닉네임) 방 생성, 방 참가, 게임 종료를 선택할 수 있다.



[그림 11] 플레이 화면

게임 화면 하단에는 좌측부터 연료통 형태의 대시 게이지 UI, 속도계 UI, 미니맵 UI를 배치하였다. 또한 화면 우측 상단에는 현재 주행 기록, 진행 중인 바퀴 수, 플레이어 순위를 표시하는 정보 UI를 구성하여 게임 진행 상황을 실시간으로 확인할 수 있도록 하였다.



[그림 12] 순위 집계 중 화면

먼저 완주한 플레이어는 순위 집계 중 화면으로 이동하여 다른 플레이어들의 경기 종료를 대기한다.



[그림 13] 경기 결과 화면

모든 플레이어가 완주하면 최종 순위를 집계한 후 경기 결과 화면을 출력한다.

4.4 문제 해결을 위한 연락처:

남윤진 - nyj121@tukorea.ac.kr

박동수 - qkrdlf8219@tukorea.ac.kr